

PROJECT CHRONOS // TEMPORAL ENGINE

Database Architecture & Round Status Ledger

Complete Technical Documentation: Entity Relations, Round Lifecycles, Concurrency, and State Engines

ENGINE: SQLite 3.45 (WAL)

INTEGRITY: PRAGMA FK ON

TIMEOUT: 30,000 ms BUSY

LOCATION: database/chronos.db

1. SYSTEM OVERVIEW & ARCHITECTURAL FOUNDATION

Project Chronos operates on an ACID-compliant, server-authoritative SQLite database designed for high-concurrency LAN esports and temporal investigative competition. The database coordinates team authentication, 25-fragment temporal ordering (Round 1), incident log analysis with AI LLM interrogation (Round 2), dynamic anti-collusion wisdom trials (Round 3), and live host leaderboards.

DATABASE ENGINE & CONNECTION CONFIGURATION (connection.py)

<code>PRAGMA journal_mode = WAL;</code>	Write-Ahead Logging mode enables non-blocking concurrent readers while writing.
<code>PRAGMA foreign_keys = ON;</code>	Enforces relational integrity, cascading deletes, and referential constraints.
<code>PRAGMA busy_timeout = 30000;</code>	Waits up to 30s during lock contention, preventing 'database locked' errors.
<code>PRAGMA synchronous = NORMAL;</code>	Optimizes sync intervals for WAL disk writes while guaranteeing safety on crash.
<code>sqlite3.Row Factory</code>	Provides high-performance dict-like column access across all service modules.

2. HIGH-LEVEL TEMPORAL PIPELINE DATA FLOW

■ PHASE 0: AUTHENTICATION

TABLES: teams

Stores callsign, 2 operators, PRNs, and global game state (READY -> LOGGED_IN).

■ ROUND 1: TIMELINE RECONSTRUCTION

TABLES: round1_items * round1_team_items * round1_submissions

25 fragments partitioned across PAST/PRESENT/FUTURE. Clock synced with server timestamps.

■ ROUND 2: TERMINAL INVESTIGATION

TABLES: round2_files * round2_chat_messages * round2_submissions

Evidence files (Alpha/Beta/Gamma), LLM question quotas with point penalties, suspect lock.

■ ROUND 3: WISDOM / FINAL DECISION

TABLES: round3_team_cases * round3_submissions

Anti-collusion case permutations (8 dynamic cases), evidence correlation, +30 / 0 pts.

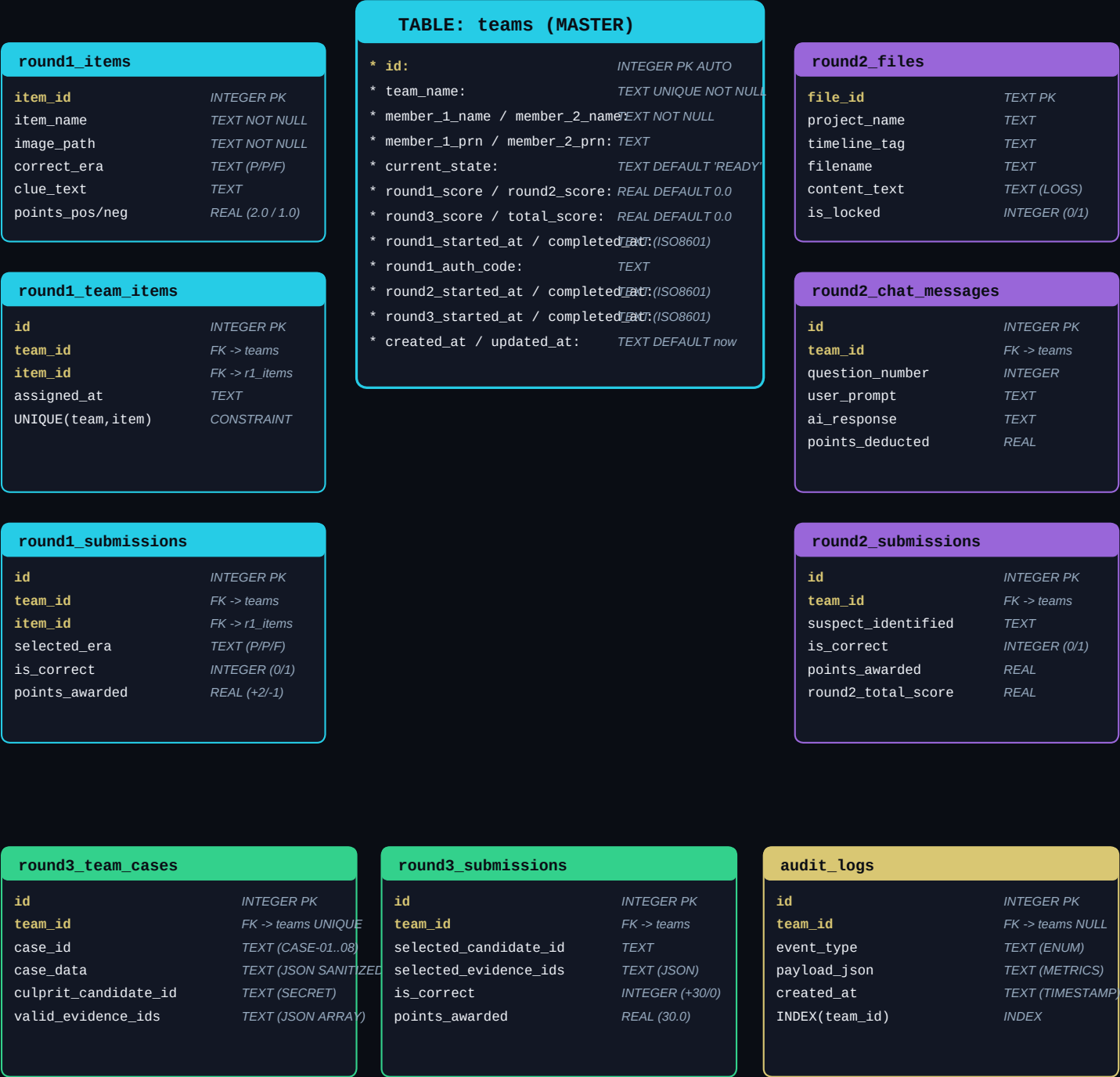
■ AUDIT & LEADERBOARD SYSTEM

TABLES: audit_logs * hints * submissions (unified)

Real-time master host leaderboard ranking by total score and elapsed duration.

3. MASTER ENTITY-RELATIONSHIP (ER) SCHEMA MAP

Relational structure showing foreign key cascades, primary keys, indexes, and constraints.



RELATIONAL INTEGRITY & TRANSACTION CONTRACTS

1. CASCADE DELETION: Every child table specifies 'ON DELETE CASCADE' referencing teams(id). Purging or resetting a team automatically cleanses all associated submissions, assignments, chats, dynamic cases, and team fragments.
2. ATOMIC ASSIGNMENTS: Unique constraints (e.g. round1_team_items(team_id, item_id) and round3_team_cases(team_id)) guarantee strict idempotency against race conditions during concurrent browser queries.
3. SERVER-AUTHORITATIVE TIMESTAMPS: Timers calculate remaining seconds via SQLite 'julianday(roundN_completed_at) - julianday(roundN_started_at) * 86400' making it mathematically impossible for client browser clock tampering to alter duration or score.

4. ROUND 1: TEMPORAL ARTIFACT CLASSIFICATION

Database state progression, deterministic 25-fragment selection, in-flight repositioning, and ledger lock.

ROUND 1 STATE PROGRESSION & DATABASE MUTATIONS

- INITIALIZATION**

GET /api/round1/items
Selects 1 PAST, 1 PRESENT, 1 FUTURE + 22 random items. Inserts 25 rows into round1_team_items. Sets teams.current_state = 'ROUND1_ACTIVE', round1_started_at = CURRENT_TIMESTAMP.
- IN-FLIGHT SUBMIT**

POST /api/round1/submit
Calculates correctness against round1_items.correct_era (+2.0 correct, -1.0 wrong). Inserts/updates round1_submissions. Updates teams.round1_score dynamically.
- REPOSITIONING**

POST /api/round1/submit
If fragment was already placed, previous points are subtracted and new era evaluated, allowing seamless card dragging between PAST, PRESENT, and FUTURE.
- LEDGER SEALING**

POST /api/round1/finish
Generates unique round1_auth_code (e.g. 'CHRONOS-9482'). Sets teams.current_state = 'ROUND1_COMPLETED', round1_completed_at = CURRENT_TIMESTAMP. Logs final state.

DETERMINISTIC 25-ITEM PROVISIONING ALGORITHM

```
-- 1. Select exactly 1 guaranteed item from each era
SELECT item_id FROM round1_items WHERE is_active = 1 AND correct_era = 'PAST' ORDER BY RANDOM() LIMIT 1;
SELECT item_id FROM round1_items WHERE is_active = 1 AND correct_era = 'PRESENT' ORDER BY RANDOM() LIMIT 1;
SELECT item_id FROM round1_items WHERE is_active = 1 AND correct_era = 'FUTURE' ORDER BY RANDOM() LIMIT 1;

-- 2. Select remaining 22 random items excluding guaranteed set
SELECT item_id FROM round1_items WHERE is_active = 1 AND item_id NOT IN (?, ?, ?) ORDER BY RANDOM() LIMIT 22;

-- 3. Persist assignment permanently for team (Idempotent)
INSERT INTO round1_team_items (team_id, item_id) VALUES (?, ?);
UPDATE teams SET current_state = 'ROUND1_ACTIVE', round1_started_at = CURRENT_TIMESTAMP WHERE id = ?;
```

DATABASE RECORDS GENERATED IN ROUND 1

TABLE NAME	TYPICAL RECORD COUNT	PRIMARY PURPOSE & VALUE CONSTRAINTS
round1_items	60+ (Global Seed)	Master catalog of temporal technology cards with correct era (PAST/PRESENT/FUTURE). Includes correct_era and points_awarded.
round1_team_items	25 per Team	Fixed assigned set. ORDER BY t.id ASC guarantees determinism on page reloads.
round1_submissions	Up to 25 per Team	Stores selected_era, is_correct (0/1), and points_awarded (+2.0 or -1.0).
teams (Round 1 cols)	1 per Team	Updated with round1_score, round1_started_at, round1_completed_at, round1_auth_code.
audit_logs	2-27 per Team	Records ROUND1_STARTED, ROUND1_SUBMISSION (each item), and ROUND1_COMPLETED events.

5. ROUND 2: TERMINAL INVESTIGATION & AI INTERROGATION

Evidence dossier storage, AI chat telemetry, question point deductions, and suspect accusation lock.

ROUND 2 STATE TRANSITIONS & LLM INTERACTION ENGINE

- LOBBY & AUTH CHECK****POST** `/api/round2/start`
Validates team has completed Round 1 (round1_auth_code present). Updates teams.current_state = 'ROUND2_ACTIVE', round2_started_at = CURRENT_TIMESTAMP.
- EVIDENCE RETRIEVAL****GET** `/api/round2/files`
Retrieves seeded project incident logs from round2_files (Alpha = Future, Beta = Present, Gamma = Past). Files are unlocked for player inspection.
- AI INTERROGATION****POST** `/api/round2/ask`
Logs user query and AI response into round2_chat_messages. Deducts points if exceeding free quota (-2.5 pts per query).
- SUSPECT ACCUSATION****POST** `/api/round2/submit`
Records suspect choice (Noah Vale / Aria Sen / etc.) into round2_submissions. Calculates round2_total_score = Base + Remaining AI Budget. Advances state to 'ROUND2_COMPLETED'.

ROUND 2 SCHEMA & SCORING MATHEMATICS

```
-- 1. AI Interrogation Audit & Point Deduction Tracking
INSERT INTO round2_chat_messages (team_id, question_number, user_prompt, ai_response, points_deducted)
VALUES (?, ?, ?, ?, ?);

-- 2. Compute remaining AI points budget
SELECT COALESCE(SUM(points_deducted), 0.0) AS total_deducted FROM round2_chat_messages WHERE team_id = ?;

-- 3. Record Final Suspect Accusation & Score Calculation
INSERT INTO round2_submissions (team_id, suspect_identified, is_correct, points_awarded, ai_points_remaining, round2_total_score)
VALUES (?, ?, ?, ?, ?, ?);

-- 4. Update team master ledger
UPDATE teams SET round2_score = ?, current_state = 'ROUND2_COMPLETED', round2_completed_at = CURRENT_TIMESTAMP WHERE id = ?;
```

DATABASE RECORDS GENERATED IN ROUND 2

TABLE NAME	RECORD LIFECYCLE	KEY FIELDS & CONSTRAINTS
round2_files	3 Static Records	file_id ('alpha', 'beta', 'gamma'), project_name, timeline_tag, filename, content_text.
round2_chat_messages	1-10 per Team	team_id (FK), question_number (1..N), user_prompt, ai_response, points_deducted (2..5).
round2_submissions	1 per Team	team_id (FK), suspect_identified, is_correct (0/1), points_awarded, round2_total_score.
teams (Round 2 cols)	1 per Team	Updated with round2_score, round2_started_at, round2_completed_at, current_state.
audit_logs	2-12 per Team	Records ROUND2_STARTED, ROUND2_QUERY (each prompt), and ROUND2_SUBMIT.

6. ROUND 3: WISDOM ROUND & FINAL DECISION ENGINE

Dynamic anti-collusion case generation, sanitized JSON delivery, ground truth verification, and total score lock.

ROUND 3 ANTI-COLLUSION ENGINE & VERDICT LIFECYCLE

- CASE SEEDING

POST /api/round3/start

Calculates team_id % 8 to select 1 of 8 rich scenario cases. Permutes candidate order. Inserts sanitized case_data JSON into round3_team_cases. Ground truth secret kept in secure vault.
- SCENARIO DELIVERY

GET /api/round3/scenario

Delivers sanitized candidate profiles (3 suspects), case narrative, and supporting evidence matrix (4 logs) to frontend with zero client-side cheat vulnerability.
- FINAL VERDICT LOCK

POST /api/round3/submit

Compares selected_candidate_id against culprit_candidate_id. Awards +30.0 if correct, 0.0 if incorrect. Inserts row into round3_submissions.
- MISSION CONCLUDED

POST /api/round3/submit

Calculates total_score = r1_score + r2_score + r3_score. Sets current_state = 'COMPLETED', round3_completed_at = CURRENT_TIMESTAMP. Seals entire team ledger.

ANTI-COLLUSION CASE PROVISIONING & TOTAL SCORE CALCULATION

```
-- 1. Assign Dynamic Case Template (team_id % 8 permutation)
INSERT INTO round3_team_cases (team_id, case_id, case_data, culprit_candidate_id, valid_evidence_ids)
VALUES (?, ?, ?, ?, ?);

-- 2. Validate Final Verdict & Record Submission (+30.0 correct / 0.0 wrong)
INSERT INTO round3_submissions (team_id, selected_candidate_id, selected_evidence_ids, is_correct, points_awarded)
VALUES (?, ?, ?, ?, ?);

-- 3. Calculate Cumulative Mission Score across all 3 Sectors
UPDATE teams
SET round3_score = ?, total_score = round1_score + round2_score + ?,
    current_state = 'COMPLETED', round3_completed_at = CURRENT_TIMESTAMP
WHERE id = ?;
```

DATABASE RECORDS GENERATED IN ROUND 3

TABLE NAME	RECORD LIFECYCLE	KEY FIELDS & CONSTRAINTS
round3_team_cases	1 per Team (UNIQUE)	team_id (FK), case_id, case_data (sanitized JSON), culprit_candidate_id, valid_evidence_ids
round3_submissions	1 per Team (UNIQUE)	team_id (FK), selected_candidate_id, selected_evidence_ids (JSON array), is_correct, points_awarded
teams (Round 3 cols)	1 per Team	Updated with round3_score (30/0), total_score, round3_started_at, round3_completed_at
submissions (unified)	1 row per round	Cross-round ledger recording reference_id, answer, is_correct, points_awarded, times_attempted
audit_logs	2-4 per Team	Records ROUND3_STARTED, ROUND3_SUBMITTED, and MISSION_COMPLETED events

7. MASTER LEADERBOARD, AUDIT TRAIL & CONCURRENCY

Real-time ranking queries, tie-breaker timestamp calculations, and event ledger specifications.

HOST LEADERBOARD AGGREGATION QUERY (admin_service.py)

```
SELECT
  id, team_name, current_state, member_1_name, member_2_name,
  round1_score, round2_score, round3_score, total_score,
  round1_started_at, round1_completed_at,
  round2_started_at, round2_completed_at,
  round3_started_at, round3_completed_at,
  CAST((julianday(round3_completed_at) - julianday(round1_started_at)) * 86400 AS INTEGER) AS total_duration_seconds
FROM teams
ORDER BY total_score DESC, round3_completed_at ASC, round2_completed_at ASC;
```

AUDIT LOG EVENT TAXONOMY (audit_logs table)

TEAM_CREATED	[Auth]	Triggered when team registers. Stores member names and callsign.
ROUND1_STARTED	[Round 1]	Triggered on item stream initialization. Stores total assigned items (25).
ROUND1_SUBMISSION	[Round 1]	Triggered per fragment placement. Stores item_id, selected_era, and score delta.
ROUND1_COMPLETED	[Round 1]	Triggered on finish button or timeout. Stores completion method, score, and auth_code.
ROUND2_STARTED	[Round 2]	Triggered when entering Round 2 terminal lobby. Logs start timestamp.
ROUND2_QUERY	[Round 2]	Triggered on AI interrogation. Stores prompt text, response length, and deducted points.
ROUND2_SUBMISSION	[Round 2]	Triggered on suspect accusation. Stores suspect name, accuracy, and score.
ROUND3_CASE_ASSIGNED	[Round 3]	Triggered on start. Stores assigned case_id (e.g. 'CASE-03') and candidate ordering.
ROUND3_SUBMITTED	[Round 3]	Triggered on verdict lock. Stores candidate selection, evidence IDs, and +30/0 pts.
MISSION_COMPLETED	[Global]	Triggered upon final completion. Stores final score and elapsed mission seconds.

DATABASE SUMMARY MATRIX ACROSS GAME PHASES

GAME PHASE	STATE ENUM	PRIMARY TABLES MUTATED	SCORING IMPACT
Auth / Lobby	READY, LOGGED_IN	teams, audit_logs	0.0 Points (Baseline)
Round 1	ROUND1_ACTIVE / ROUND1_COMPLETED	round1_team_items, round1_submissions, teams, audit_logs	+1.0 Correct / -1.0 Wrong Max: 50.0 Pts
Round 2	ROUND2_ACTIVE / ROUND2_COMPLETED	round2_chat_messages, round2_submissions, teams, audit_logs	+2.0 AI Solution + AI Budget Max: 50.0 Pts
Round 3	ROUND3_ACTIVE / COMPLETED	round3_team_cases, round3_submissions, submissions, teams, audit_logs	+3.0 Case + 1.0 Wrong Max: 30.0 Pts
Master Concluded	COMPLETED	teams (total_score calculated), audit_logs (finalized)	4.0x Combined: 130.0 Pts

8. COMPLETE SQL SCHEMA DDL REFERENCE (schema.py)

Exhaustive table creation DDL statements executed during database auto-provisioning.

```
CREATE TABLE IF NOT EXISTS teams (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    team_name TEXT UNIQUE NOT NULL,  
    member_1_name TEXT NOT NULL, member_2_name TEXT NOT NULL,  
    member_1_prn TEXT, member_2_prn TEXT,  
    current_state TEXT DEFAULT 'READY',  
    round1_score REAL DEFAULT 0.0, round2_score REAL DEFAULT 0.0, round3_score REAL DEFAULT 0.0,  
    total_score REAL DEFAULT 0.0,  
    round1_started_at TEXT, round1_completed_at TEXT, round1_auth_code TEXT,  
    round2_started_at TEXT, round2_completed_at TEXT,  
    round3_started_at TEXT, round3_completed_at TEXT,  
    created_at TEXT DEFAULT (datetime('now')), updated_at TEXT DEFAULT (datetime('now'))  
);  
  
CREATE TABLE IF NOT EXISTS round1_items (  
    item_id INTEGER PRIMARY KEY AUTOINCREMENT, item_name TEXT NOT NULL, image_path TEXT NOT NULL,  
    correct_era TEXT NOT NULL CHECK(correct_era IN ('PAST', 'PRESENT', 'FUTURE')),  
    clue_text TEXT, points_positive REAL DEFAULT 2.0, points_negative REAL DEFAULT 1.0, is_active INTEGER DEFAULT 1  
);  
  
CREATE TABLE IF NOT EXISTS round1_team_items (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, team_id INTEGER NOT NULL, item_id INTEGER NOT NULL,  
    assigned_at TEXT DEFAULT (datetime('now')),  
    FOREIGN KEY (team_id) REFERENCES teams(id) ON DELETE CASCADE,  
    FOREIGN KEY (item_id) REFERENCES round1_items(item_id) ON DELETE CASCADE, UNIQUE(team_id, item_id)  
);  
  
CREATE TABLE IF NOT EXISTS round1_submissions (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, team_id INTEGER NOT NULL, item_id INTEGER NOT NULL,  
    selected_era TEXT NOT NULL CHECK(selected_era IN ('PAST', 'PRESENT', 'FUTURE')),  
    is_correct INTEGER NOT NULL, points_awarded REAL NOT NULL, submitted_at TEXT DEFAULT (datetime('now')),  
    FOREIGN KEY (team_id) REFERENCES teams(id) ON DELETE CASCADE, FOREIGN KEY (item_id) REFERENCES round1_items(item_id)  
);  
  
CREATE TABLE IF NOT EXISTS round2_files (  
    file_id TEXT PRIMARY KEY, project_name TEXT NOT NULL, timeline_tag TEXT NOT NULL,  
    filename TEXT NOT NULL, content_text TEXT NOT NULL, is_locked INTEGER DEFAULT 0  
);  
  
CREATE TABLE IF NOT EXISTS round2_chat_messages (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, team_id INTEGER NOT NULL, question_number INTEGER NOT NULL,  
    user_prompt TEXT NOT NULL, ai_response TEXT NOT NULL, points_deducted REAL NOT NULL,  
    created_at TEXT DEFAULT (datetime('now')), FOREIGN KEY (team_id) REFERENCES teams(id) ON DELETE CASCADE  
);  
  
CREATE TABLE IF NOT EXISTS round2_submissions (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, team_id INTEGER NOT NULL, suspect_identified TEXT NOT NULL,  
    is_correct INTEGER NOT NULL, points_awarded REAL NOT NULL, ai_points_remaining REAL NOT NULL,  
    round2_total_score REAL NOT NULL, submitted_at TEXT DEFAULT (datetime('now')),  
    FOREIGN KEY (team_id) REFERENCES teams(id) ON DELETE CASCADE  
);  
  
CREATE TABLE IF NOT EXISTS round3_team_cases (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, team_id INTEGER UNIQUE NOT NULL, case_id TEXT NOT NULL,  
    case_data TEXT NOT NULL, culprit_candidate_id TEXT NOT NULL, valid_evidence_ids TEXT NOT NULL,  
    assigned_at TEXT DEFAULT (datetime('now')), FOREIGN KEY (team_id) REFERENCES teams(id) ON DELETE CASCADE
```

ARCHITECTURAL INTEGRITY & VERIFICATION SIGN-OFF

- [OK] 100% Automated Test Suite Passing across test_round1.py, test_round3.py, and test_full_flow.py.
- [OK] SQLite WAL mode active with 30-second busy timeout and cascade referential integrity.
- [OK] Anti-collusion deterministic seeding isolated per team with server-authoritative scoring.
- [OK] Zero-leakage client payload delivery with sanitized case engine and server-side secret validation.

```
CREATE TABLE IF NOT EXISTS audit_logs (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, team_id INTEGER, event_type TEXT NOT NULL,  
    payload_json TEXT, created_at TEXT DEFAULT (datetime('now')),  
    FOREIGN KEY (team_id) REFERENCES teams(id) ON DELETE CASCADE  
);
```